

TP Temps Reel - Conception d'un ECU

Regulateur de vitesse

Jean-Loup Chrestien de Beauminy

Dolovan Akdag

16 juin 2026

Support : ESP32 (dual core) + FreeRTOS, sous ESP-IDF. Code dans le dossier `ecu/`.

Table des matières

1	Choix d'architecture	2
2	Tableau des taches	2
3	Diagramme des taches et evenements	2
4	Justification des mecanismes FreeRTOS	3
5	Charge cpu theorique	3
6	Priorite de l'uart par rapport au controle moteur	4
7	Choix des priorites et preemption	4
8	Monitoring et debug	4
9	Analyse de robustesse	5
9.1	Test de la couche protocole sur pc	5
9.2	Test du firmware complet sous emulation QEMU	6
10	Limitations	6

1 Choix d'architecture

L'idée de depart vient du cours : une tache = une responsabilite. Plutot qu'une grosse boucle qui lit l'uart, calcule le pid et emet tout au meme endroit (ce qui serait monolithique et penalise), j'ai decoupe le systeme en quatre taches qui communiquent uniquement par des primitives FreeRTOS, jamais par des variables globales partagees a nu.

Les quatre taches :

- **rx** : lit le bus uart et decode les trames (machine a etats octet par octet). c'est le seul point d'entree des donnees.
- **control** : le regulateur pid, reveille toutes les 100 ms. il consomme les consignes et vitesses decodees par rx et emet la commande moteur.
- **stats** : la telemetrie, une fois par seconde. lit les compteurs et emet la trame stats.
- **failsafe** : la securite. normalement endormie, reveillee par un evenement (front gpio ou timeout watchdog).

Le point cle de l'archi c'est l'isolation sur les deux coeurs de l'esp32. Le parser (rx) est epingle sur le **core 0**, et tout ce qui est critique pour la regulation (pid, failsafe, stats) sur le **core 1**. Comme ca un flood de messages sur l'uart sature au pire le core 0, mais le pid sur le core 1 continue de tourner a l'heure. C'est la reponse directe a l'exigence non fonctionnelle 02 (isolation des defaillances).

La communication rx vers control passe par une **queue** de commandes. rx pousse des couples (**type**, **valeur**), control les depile au debut de chaque cycle. On a donc un producteur / consommateur classique : rx ne touche jamais aux variables internes du pid, et control n'a pas a savoir comment une trame est decodee. Si la queue est pleine (control en retard, ne devrait pas arriver) on jette la commande et on incremente un compteur, on ne bloque pas rx.

L'etat de supervision (mode courant, flag failsafe) est lu par control et ecrit par rx (changement de mode) et par failsafe. Ces acces sont tres courts donc proteges par une section critique (spinlock), pas par un mutex : on parle de quelques instructions, prendre un mutex couterait plus cher que le travail lui meme.

2 Tableau des taches

nom	prio	coeur	periodicite	stack (octets)
failsafe	4	1	evenementielle	2048
control	3	1	periodique 100 ms	2560
rx	2	0	bloquante sur uart	3072
stats	1	1	periodique 1000 ms	3072

Les priorites vont de 1 (idle+1) a 4. failsafe est la plus haute pour garantir son temps de reponse, stats la plus basse car non critique. Les tailles de stack sont dimensionnees a la louche puis verifiables avec `uxTaskGetStackHighWaterMark`. rx et stats ont un peu plus de marge parce qu'elles font des logs et manipulent des buffers.

3 Diagramme des taches et evenements

Evenements possibles :

- arrivee d'octets sur l'uart : rx debloque, decode, eventuellement push une commande dans la queue et rearme le watchdog.
- tick 100 ms : control se reveille (`vTaskDelayUntil`), depile la queue, calcule le pid, emet output (uniquement en mode auto).
- tick 1 s : stats emet les compteurs.
- 2 s sans trame valide : le timer expire, son callback notifie failsafe.

- front descendant sur le gpio d'erreur : l'isr notifie failsafe.
- reception d'un mode_set : rx remet le mode et efface le flag failsafe (reprise, exigence 06).

La justification des priorites est detaillee aux sections 6 et 7.

4 Justification des mecanismes FreeRTOS

- **queue** pour rx vers control : decouple le producteur du consommateur et lisse les rafales. control lit a son rythme, rx n'est jamais bloque par le pid. envoi non bloquant (timeout 0) pour ne pas ralentir le parser si la queue se remplit.
- **mutex** sur l'emission uart : trois taches peuvent emettre (control, stats, failsafe). sans protection, deux ecritures pourraient entrelacer leurs octets et casser une trame (interdit par nf03). le mutex serialise l'emission. j'ai pris un mutex et pas un semaphore binaire exactement pour l'**heritage de priorite** : si stats (p1) tient le mutex et que failsafe (p4) le demande, stats est temporairement remontee a p4 le temps de liberer, donc pas d'inversion de priorite non maitrisee (nf04).
- **section critique** / **spinlock** pour les compteurs et l'etat de supervision : acces de quelques cycles, sur un dual core il faut un spinlock (pas juste masquer les interruptions) pour rester coherent entre les deux coeurs. plus rapide qu'un mutex pour un increment.
- **software timer** pour le watchdog rx : un timer one-shot de 2 s rearme a chaque trame valide. s'il expire c'est qu'on n'a rien recu, son callback notifie failsafe. pas besoin d'une tache dediee qui boucle.
- **task notification** pour reveiller failsafe : le moyen le plus rapide et le plus leger de signaler une tache depuis une isr ou un callback (plus rapide qu'un semaphore). la valeur de la notification transporte la cause (gpio ou timeout).
- **vTaskDelayUntil** pour control et stats : maintient une periode absolue sans derive. un simple vTaskDelay accumulerait le temps de calcul a chaque tour et ferait deriver la periode, donc du jitter. vTaskDelayUntil cale les reveils sur une base de temps fixe.

5 Charge cpu theorique

Le tick FreeRTOS est regle a 1000 Hz (1 ms) dans `sdkconfig.defaults`. A 100 Hz par default la resolution serait de 10 ms, impossible de garantir un jitter sous 5 ms. A 1 ms le reveil du pid est cale au tick pres.

Les wcet ci-dessous sont des estimations (pas de mesure oscillo, build only).

core 1 :

- control : depiler la queue + calcul pid + construire et copier une trame output dans le ring buffer tx. estime 200 us. charge = 200 us / 100 ms = **0,2 %**.
- stats : snapshot des compteurs + une trame + un log console. estime 500 us toutes les secondes = **0,05 %**.
- failsafe : evenementiel et rare, negligeable.

Total core 1 : largement sous **1 %**.

core 0 :

- rx : a 115200 bauds en 8n1 on a au plus 115200 / 10 = 11520 octets/s. chaque octet passe dans le parser (un switch, quelques instructions, environ 0,5 us). ca fait environ 6 ms de traitement par seconde, plus le cout des lectures uart et des interruptions du driver. on reste autour de **1 a 2 %** meme en flood continu.

Conclusion : meme sous surcharge uart maximale, le core 1 (la regulation) n'est quasiment pas charge. On est tres loin de la saturation, la marge est enorme. Le determinisme du pid est garanti par construction.

6 Priorite de l'uart par rapport au controle moteur

C'est la question centrale du sujet. Si on traitait la reception uart en priorite haute, un flood de messages monopoliserait le cpu et retarderait le calcul du pid, donc la periode de 100 ms ne serait plus tenue et le jitter exploserait.

J'ai fait l'inverse : **control (pid) est en priorite 3, strictement au dessus de rx (parser) en priorite 2**. Donc des qu'un cycle pid doit s'executer, il preempt le parser, calcule, emet, et rend la main. Le flood attend. En plus, les deux taches sont sur des coeurs differents (rx core 0, control core 1), donc en pratique elles ne se disputent meme pas le cpu : le parser peut tourner a fond sur le core 0 sans jamais grignoter une microseconde au pid sur le core 1.

La reception n'est pas sacrifiee pour autant : la queue absorbe les rafales, et la perte eventuelle de messages sous flood extreme est comptee (compteur dropped) et non bloquante. On privilegie la regulation, qui est la fonction critique, et on degrade proprement la reception si besoin.

7 Choix des priorites et preemption

L'ordonnanceur est en preemptif a priorite fixe (fpp). L'ordre des priorites suit la criticite temps reel :

1. **failsafe (4, max)** : la securite passe avant tout. comme elle est la plus prioritaire, des qu'elle est notifiee elle preempt n'importe quelle autre tache et s'execute tout de suite. c'est ce qui garantit le temps de reponse sous 5 ms exige : elle force output=0 et emet l'alarme sans attendre le prochain cycle du pid. le travail dans l'isr gpio est reduit au minimum (juste une notification, le reste est fait dans la tache).
2. **control (3)** : la regulation, fonction principale, doit etre prioritaire sur tout sauf la securite.
3. **rx (2)** : important mais ne doit pas perturber control.
4. **stats (1)** : telemetrie, peut attendre, aucune contrainte dure.

La preemption joue donc dans ce sens : failsafe > control > rx > stats. Un evenement de securite interrompt un calcul pid en cours ; un calcul pid interrompt le parser ; le parser passe avant la telemetrie. C'est exactement l'ordre voulu pour respecter les contraintes.

8 Monitoring et debug

- **trame stats (0x83)** emise sur le bus toutes les secondes : rx valides, rx crc erreur, rx dropped, tx output, et un compteur en plus rx unknown (ids inconnus). l'outil de test cote pc peut suivre l'etat de l'ecu en direct.
- **log console** sur l'uart0 (separe du bus, qui est sur uart2) : chaque seconde la tache stats sort une ligne avec le mode courant, le flag failsafe, tous les compteurs et le **heap libre** (`esp_get_free_heap_size`) pour detecter une fuite memoire.
- **log de warning** a chaque failsafe avec la cause.
- **led** sur le gpio 2 (led onboard) : allumee en failsafe, eteinte des la reprise. debug visuel immediat sans brancher de console.
- separer le bus binaire (uart2) des logs texte (uart0) evite de polluer les trames avec des messages de debug, et c'est lisible cote console.
- en debug plus pousse, `uxTaskGetStackHighWaterMark` permet de verifier le dimensionnement des stacks et `vTaskGetRunTimeStats` de mesurer la charge reelie par tache.

9 Analyse de robustesse

Le sujet decrit un script de stress qui injecte : des trames collees, des trames fragmentees, des crc faux, des ids inconnus, et un flood. La robustesse repose entierement sur le parser, qui est une machine a etats (start, len, data, crc) qui garde son etat entre deux lectures uart.

- **trames fragmentees** : comme on decode octet par octet et qu'on conserve l'etat du parser entre deux appels a la lecture uart, une trame coupee au milieu reprend exactement la ou elle s'etait arreee. pas de perte d'etat.
- **trames collees** : apres avoir traite une trame complete on repart directement a l'etat start. l'octet suivant est aussitot interprete comme le debut de la trame d'apres. on enchaîne sans rien perdre.
- **crc faux** : on recalcule le xor (len + type + payload) et on compare a l'octet reçu. si ca ne colle pas, la trame est jetee silencieusement, le compteur crc augmente et on resynchronise.
- **len incoherente** : si le len annonce est nul ou depasse la taille max acceptee, on rejette et on repart a start sans rien stocker. ca evite qu'une longueur absurde fasse deborder le buffer ou bloque le parser sur une trame qui n'arrivera jamais.
- **id inconnu** : trame structurellement valide mais type non gere, ignoree, le compteur unknown augmente (le crc etait bon, donc on ne le compte pas en erreur crc). on ne plante pas, on continue.
- **flood** : le parser tourne sur le core 0 sans toucher au pid (core 1). la queue absorbe les rafales et, si elle sature, on droppe en comptant. la regulation reste a l'heure.

Cote securite, les deux declencheurs de failsafe sont independants : si le pc arrete d'emettre, le watchdog de 2 s tombe et coupe ; un front sur le gpio d'erreur coupe aussi, en moins de 5 ms grace a la tache prioritaire. apres, un simple mode_set en auto relance le tout.

9.1 Test de la couche protocole sur pc

J'ai teste a deux niveaux : d'abord un test unitaire de la couche protocole sur pc, puis le firmware complet sous emulation qemu avec le vrai script de stress. Pour le test unitaire j'ai isole la couche protocole (construction de trame, crc, et la meme machine a etats que dans le firmware) et je l'ai compilee et executee sur pc avec les memes cas que le script : trame propre, trames collees, trame fragmentee, crc casse, id inconnu. La compilation passe avec `-Wall -Wextra -Werror` et le decodage se comporte comme prevu.

```
== test couche protocole ==

1. setpoint 90.0 -> trame de 9 octets : aa 05 00 01 00 00 b4 42 f2
   -> trame valide type=0x01 val=90.00

2. deux trames collees (setpoint + speed)
   -> trame valide type=0x01 val=90.00
   -> trame valide type=0x02 val=42.50

3. trame fragmentee (2 morceaux)
   -> trame valide type=0x02 val=42.50

4. trame avec crc casse
   -> crc faux (recu 0x0d attendu 0xf2) rejet

5. id inconnu (0x42)
   -> id inconnu 0x42 ignore

== bilan : ok=4 unknown=1 rejets=1 ==
```

Listing 1 – sortie reelle du test protocole (gcc, sur pc)

On retrouve bien les six comportements attendus : la trame valide est decodee, les deux trames collees sont separees correctement, la trame fragmentee est reconstituee, le crc casse est rejete, et l'id inconnu est ignore sans planter.

9.2 Test du firmware complet sous emulation QEMU

Faute de carte physique, j'ai fait tourner le firmware compile (esp-idf v5.4.4) dans l'émulateur **qemu xtensa** fourni par espressif. Le bus ecu (uart2) est expose par qemu sur un socket tcp, et le script de stress du sujet s'y connecte directement (au lieu d'un `/dev/ttyUSB0`). C'est donc le vrai scenario du sujet (regulation, flood, trames cassees, silence radio) rejoue de bout en bout, le firmware s'executant reellement.

Ce que la trace montre, point par point :

- **regulation** : en mode auto, `rxok` et `out` montent regulierement, la commande moteur est emise a chaque cycle.
- **flood** : les 50 trames a id aleatoire sont comptees en `unk=50` (ids inconnus), sans planter et sans bloquer la regulation.
- **trames cassees** : les deux trames volontairement invalides (crc faux et fragment mal termine) donnent `crc=2`.
- **aucune perte** : `drop=0` sur tout le run, la queue n'a jamais deborde meme pendant le flood.
- **failsafe** : a la coupure des emissions, l'ecu passe en `fs=1`, force `output=0` et emet l'alarme. le script cote pc confirme (**verification reussie**).
- **pas de fuite memoire** : `heap` reste a 289312 octets du debut a la fin.

La seule chose que l'emulation ne valide pas finement, c'est le timing exact (qemu n'est pas cycle-accurate), donc le jitter sous 5 ms reste a confirmer a l'oscilloscope sur une vraie carte.

10 Limitations

- **pas de carte physique** : le firmware compile sous esp-idf v5.4.4 (`idf.py set-target esp32 && idf.py build`) et tourne sous emulation qemu, mais n'a pas ete teste sur un vrai esp32. comme qemu n'est pas cycle-accurate, le **jitter sous 5 ms** et le temps de reaction du failsafe restent a mesurer a l'oscilloscope sur cible.
- du meme coup les **wcet et la charge cpu restent des estimations**, pas des mesures hardware. a confirmer avec `vTaskGetRunTimeStats` et l'analyse du high water mark sur une vraie carte.
- les **coefficients du pid** (`kp=1`, `ki=0.1`, `kd=0.01`) sont ceux de l'exemple du sujet, ils ne sont pas regles pour un vrai modele de vehicule. il faudrait les ajuster sur le banc.
- le bus ecu est sur **uart2** (pins 16/17), separe de la console. pour brancher le script de test il faut un adaptateur usb-serie sur ces pins, pas le port usb de debug. (on peut basculer le bus sur `uart0` et couper les logs si on veut coller au script tel quel.)
- la trame **dbg (0xff)** n'est pas emise, je m'en tiens aux messages exiges. elle pourrait servir a dumper l'etat interne du pid pour le reglage.
- pas de persistance : au reset on repart en mode off, compteurs remis a zero.

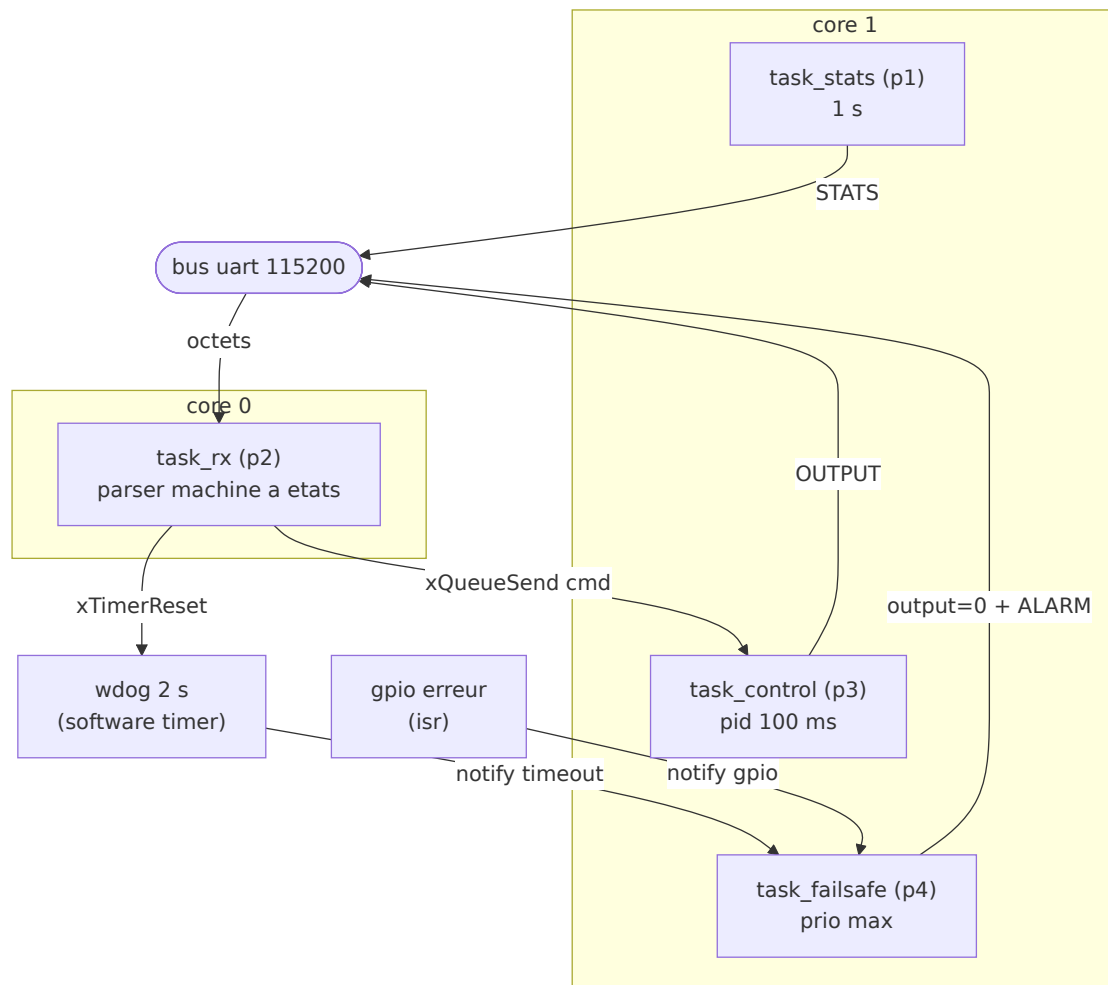


FIGURE 1 – taches, coeurs et flux d'évenements (primitives freertos en legende)

```

$ ECU_PORT=socket://localhost:5557 python tools/stress_test.py
--- debut phase normale (30s) ---
[SPEED] (31.60 km/h) | commande: 55.77
[SPEED] (44.38 km/h) | commande: 51.75
[TELEMETRIE] recue (1s)
[SPEED] (55.49 km/h) | commande: 47.50
[SPEED] (62.40 km/h) | commande: 35.08
[ ... regulation : la vitesse converge vers la consigne 90 ... ]
[SPEED] (86.92 km/h) | commande: 47.53
--- debut phase de stress ---
action: surcharge cpu (flood)...
action: envoi de trame fragmentee...
action: envoi erreur crc...
--- debut phase normale (5s) ---
[SPEED] (86.92 km/h) | commande: 47.96
[SPEED] (86.92 km/h) | commande: 0.00
[ALERTE ECU] : timeout rx
verification reussie : l'ecu a coupe la commande moteur (output=0)
test termine.

```

FIGURE 2 – cote pc : le script de stress branche sur l'ecu emule. la vitesse converge vers la consigne, le stress est encaisse, puis le failsafe coupe la commande (output=0) avec l'alarme timeout rx

```

qemu-system-xtensa -M esp32 (console uart0 de l'ecu)
I (1904) ecu: demarrage ecu
I (2916) ecu: mode=0 fs=0 | rxok=0 crc=0 drop=0 out=0 unk=0 | heap=289312
W (3921) ecu: failsafe cause=1
[ ... le script passe en mode auto et regule ... ]
I (20912) ecu: mode=2 fs=0 | rxok=37 crc=0 drop=0 out=34 unk=0 | heap=289312
I (28912) ecu: mode=2 fs=0 | rxok=117 crc=0 drop=0 out=114 unk=0 | heap=289312
I (44913) ecu: mode=2 fs=0 | rxok=275 crc=0 drop=0 out=274 unk=0 | heap=289312
I (48913) ecu: mode=2 fs=0 | rxok=313 crc=2 drop=0 out=314 unk=50 | heap=289312
W (54849) ecu: failsafe cause=1
I (55913) ecu: mode=0 fs=1 | rxok=353 crc=2 drop=0 out=374 unk=50 | heap=289312

```

FIGURE 3 – cote ecu (console uart0) : les compteurs de telemetrie pendant le run